

Object Oriented Programming with Java (BCS-403)

Unit 5: Spring Framework and Spring Boot

Complete Notes

Topics Covered

Spring Core Basics, Spring Dependency Injection concepts, Spring Inversion of Control, AOP, Bean Scopes, Singleton, Prototype, Request, Session, Application, Web Socket, Auto wiring, Annotations, Life Cycle Call backs, Bean Configuration styles, Spring Boot Build Systems, Spring Boot Code Structure, Spring Boot Runners, Logger, Building RESTful Web Services, Rest Controller, Request Mapping, Request Body, Path Variable, Request Parameter, GET, POST, PUT, DELETE APIs, and Build Web Applications.

Contents

1	Introduction to Java Frameworks	4
1.1	Why Java Frameworks are Used	4
2	Spring Framework	4
2.1	Key Features of Spring Framework	4
2.2	Spring Modules	4
3	Advantages of Spring Framework	5
4	Dependency Injection	5
4.1	Purpose of Dependency Injection	5
4.2	Types of Spring Dependency Injection	6
4.2.1	Constructor-Based Dependency Injection	6
4.2.2	Setter-Based Dependency Injection	6
4.2.3	Recommended Practice	6
5	Inversion of Control (IoC)	6
5.1	Spring IoC Container	6
5.2	Main Features of Spring IoC	7
6	Aspect Oriented Programming (AOP)	7
6.1	Cross-Cutting Concerns	7
6.2	How AOP Works	7
6.3	Dominant Frameworks in AOP	7
6.4	Spring AOP Proxy Mechanism	8
6.5	Common Terminologies in AOP	8
7	Bean Scope	8
7.1	Bean Definition	8
7.2	Types of Bean Scope	8
7.3	Additional Web-Related Scopes	8
7.3.1	Application Scope	8
7.3.2	WebSocket Scope	9
7.4	Summary Table of Bean Scopes	9
8	Autowiring in Spring	9
8.1	Key Points	9
8.2	Modes of Autowiring	9
8.3	Advantage and Disadvantage	10
9	Annotations	10
9.1	Uses of Java Annotations	10
9.2	Built-in Java Annotations	10
9.3	Annotation Basics	10
9.4	Where Annotations Can Be Used	10
9.5	Examples	11

10 Spring Bean Life Cycle and Callbacks	11
10.1 Bean Life Cycle Process	11
10.2 Initialization and Destruction Callbacks	11
11 Bean Configuration Styles	11
11.1 Annotation-Based Configuration	11
11.2 XML-Based Configuration	12
11.3 Java-Based Configuration	12
12 Spring Boot	12
12.1 Build Systems	12
12.2 Dependency Management	12
12.2.1 Maven Dependency	12
12.2.2 Gradle Dependency	12
13 Spring Boot Code Structure	13
13.1 Recommended Project Organization	13
13.2 Avoid Default Package	13
13.3 Package Naming Convention	13
13.4 Main Application Class Placement	13
13.5 Common Structuring Approaches	13
13.6 Example Main Application Class	14
14 RESTful Web Services	14
14.1 Common HTTP Methods	14
14.2 Importance of HTTP Methods	14
14.3 GET Method	14
14.4 POST Method	14
14.5 PUT Method	14
14.6 DELETE Method	15
15 Rest Controller	15
16 Request Mapping	15
17 Request Body	15
17.1 @RequestBody	15
18 Path Variable	15
19 Building Web Applications	16
19.1 Technologies Used in Web Applications	16
19.2 Need for Server-Side Technology	16
20 MVC Architecture	16
20.1 Components of MVC	16
21 Practical Task	16
21.1 Objective	16

21.2 Suggested Problem Statements	16
21.3 Functional Requirements	17
21.4 Technical Requirements	17
21.5 Implementation Steps	17
21.6 Evaluation Criteria	17
22 Quick Recap	17

1. Introduction to Java Frameworks

A Java framework is a body of predefined code used by programmers to develop applications on the web. These frameworks provide reusable classes, methods, utilities, and conventions so that developers do not need to build every feature from scratch. A framework controls many common tasks such as request handling, data processing, system communication, validation, and integration with other tools.

1.1 Why Java Frameworks are Used

It is time-consuming to build large Java applications from the ground up. Integrating a framework into the application stack speeds up the development process. Frameworks often provide built-in support for application-wide features such as:

- data retrieval,
- security,
- privacy,
- reusable development options,
- improved efficiency,
- better reliability for web applications.

2. Spring Framework

Spring Framework simplifies the development of Java EE applications. It is useful for both beginners and experienced developers. It is a lightweight framework and is often called a **framework of frameworks** because it supports integration with other technologies such as Struts, Hibernate, Tapestry, EJB, and JSF.

Spring was developed by Rod Johnson in 2003. It is widely used as a Dependency Injection framework to build loosely coupled Java applications.

2.1 Key Features of Spring Framework

- lightweight architecture,
- support for loose coupling,
- easy testing,
- modular design,
- fast development support,
- support for multiple enterprise technologies.

2.2 Spring Modules

Spring is composed of multiple modules such as:

- IoC,

- AOP,
- DAO,
- Context,
- ORM,
- Web MVC.

3. Advantages of Spring Framework

The major advantages are:

1. **Predefined Templates:** Spring provides templates for technologies such as JDBC, Hibernate, and JPA, so developers do not have to write repetitive code.
2. **Loose Coupling:** Dependency Injection reduces strong dependency between classes.
3. **Easy to Test:** Dependency Injection improves testability. Spring applications can often be tested without a full server setup.
4. **Lightweight:** Spring uses POJO-based programming and does not force programmers to inherit framework-specific classes.
5. **Fast Development:** Dependency Injection and framework support speed up Java EE development.
6. **Powerful Abstraction:** Spring provides abstraction for enterprise specifications such as JMS, JDBC, JPA, and JTA.
7. **Declarative Support:** Spring supports declarative caching, validation, transactions, and formatting.

4. Dependency Injection

Dependency Injection (DI) is a design pattern that removes dependency management from the programming code so that the application becomes easier to manage and test. DI helps create loosely coupled code.

When a Java application grows, classes should remain as independent as possible so that:

- the classes can be reused,
- each class can be tested independently,
- the code remains cleaner and easier to maintain.

Dependency Injection helps in gluing classes together while keeping them independent. The Spring Core module is responsible for injecting dependencies.

4.1 Purpose of Dependency Injection

- reduces tight coupling,
- improves code reusability,

- improves testability,
- simplifies maintenance,
- keeps business logic cleaner.

4.2 Types of Spring Dependency Injection

Spring supports two main types of dependency injection.

4.2.1 Constructor-Based Dependency Injection

Constructor-based DI is accomplished when the container invokes a class constructor with arguments, where each argument represents a dependency required by that class.

4.2.2 Setter-Based Dependency Injection

Setter-based DI is accomplished when the container calls setter methods on a bean after invoking a no-argument constructor or factory method.

4.2.3 Recommended Practice

A common rule is:

- use constructor arguments for mandatory dependencies,
- use setter methods for optional dependencies.

This keeps the code cleaner and improves decoupling because the object does not need to locate its dependencies by itself.

5. Inversion of Control (IoC)

Inversion of Control is a software engineering principle in which control of objects or program flow is transferred to a container or framework. In object-oriented programming, this means object creation and lifecycle management are handled by the framework instead of the developer.

5.1 Spring IoC Container

The Spring IoC Container is the core of Spring Framework. It:

- creates objects,
- configures objects,
- assembles dependencies,
- manages the complete bean lifecycle,
- helps make the application configurable.

The container can obtain configuration information from:

- XML configuration files,

- Java code,
- Java annotations,
- Java POJO classes.

Objects managed by the container are called **beans**.

5.2 Main Features of Spring IoC

- creating objects for the application,
- managing application objects,
- helping the application remain configurable,
- managing dependencies.

6. Aspect Oriented Programming (AOP)

Aspect Oriented Programming breaks code into different modules and treats **aspects** as the key unit of modularity. AOP is used to manage **cross-cutting concerns** such as transaction management, authentication, logging, and security without cluttering the core business logic.

6.1 Cross-Cutting Concerns

A cross-cutting concern is a concern that affects many parts of an application and should be centralized as much as possible. Common examples are:

- transaction management,
- authentication,
- logging,
- security.

6.2 How AOP Works

AOP adds additional behavior, called **advice**, to existing code. Instead of repeating code such as security checks in multiple methods, that behavior can be defined once and applied wherever required.

6.3 Dominant Frameworks in AOP

Three frameworks named are:

- **AspectJ**: A Java extension created at PARC with its own compiler and weaver.
- **JBoss**: An open-source Java application server used in Java development.
- **Spring**: Uses XML-based configuration and annotations, with AspectJ libraries commonly used for parsing and matching.

6.4 Spring AOP Proxy Mechanism

Spring uses a proxy-based mechanism. A proxy object wraps the original object and applies the relevant advice during method calls. Proxy objects may be created manually using a proxy factory bean or automatically through configuration. After execution completes, the proxy object is discarded. Proxy objects enrich the original behavior of the real object.

6.5 Common Terminologies in AOP

1. **Aspect:** A class that implements cross-cutting concerns such as transaction handling or logging. It can be a normal class configured through XML or a class annotated with `@Aspect`.
2. **Weaving:** The process of linking aspects with an advised object. Weaving can happen at compile time, load time, or runtime. Spring AOP performs weaving at runtime.

7. Bean Scope

Bean scope defines the lifecycle of a bean:

- when the bean is instantiated,
- how long the bean lives,
- how many bean objects are created.

7.1 Bean Definition

Beans are the objects that form the backbone of a Spring application. A bean is an object instantiated, assembled, and managed by the Spring IoC container.

7.2 Types of Bean Scope

1. **Singleton:** Only one instance is created for a bean definition per Spring IoC container, and the same object is shared for every request. This is the default scope.
2. **Prototype:** A new instance is created every time the bean is requested.
3. **Request:** A new instance is created for each HTTP request. This is valid only in a web-aware Spring `ApplicationContext`.
4. **Session:** A single bean instance is scoped to the lifecycle of an HTTP session.
5. **Global Session:** A single bean instance is scoped to the lifecycle of a global HTTP session in a web-aware environment.

7.3 Additional Web-Related Scopes

7.3.1 Application Scope

In application scope, the container creates one instance per web application runtime. It is similar to singleton scope, but the application-scoped bean is singleton per `ServletContext`, whereas singleton scope is singleton per `ApplicationContext`. The application-scoped bean is visible as a `ServletContext` attribute.

7.3.2 WebSocket Scope

The WebSocket protocol enables two-way communication between client and server over a single TCP connection. In WebSocket scope, beans are stored in WebSocket session attributes. The same bean instance is returned during the entire WebSocket session.

7.4 Summary Table of Bean Scopes

Scope	Description
Singleton	Only one single instance is created. Default bean scope.
Prototype	Creates any number of instances from a single bean configuration.
Request	Bean instance is limited to one HTTP request lifecycle.
Session	Bean instance is limited to one HTTP session.
Global Session	Bean instance is limited to a global session context.
Application	One instance per web application runtime or <code>ServletContext</code> .
WebSocket	One bean instance for the WebSocket session.

8. Autowiring in Spring

Autowiring allows Spring to inject dependencies automatically. The Spring container detects dependencies and relationships between beans and injects them without requiring explicit wiring code in many cases.

8.1 Key Points

- Autowiring is enabled using the `@Autowired` annotation.
- It reduces the amount of code needed for dependency injection.
- It offers less control to the programmer compared with manual configuration.

8.2 Modes of Autowiring

Mode	Description
No	Autowiring is disabled. Dependency injection is not performed automatically.
byName	Injects dependency according to bean name.
byType	Injects dependency according to bean type.
Constructor	Injects required dependencies by invoking the constructor.
Autodetect	Uses automatic detection by combining other modes.

8.3 Advantage and Disadvantage

- **Advantage:** Less code because explicit injection code is reduced.
- **Disadvantage:** Less control for the programmer.

9. Annotations

Annotations are metadata that provide extra information about a program. They do not directly change the execution of compiled code.

9.1 Uses of Java Annotations

Java annotations are used mainly for:

- compiler instructions,
- build-time instructions,
- runtime instructions.

9.2 Built-in Java Annotations

Three built-in annotations listed are:

- `@Deprecated`,
- `@Override`,
- `@SuppressWarnings`.

9.3 Annotation Basics

- A Java annotation starts with the `@` symbol followed by the annotation name.
- The `@` symbol tells the compiler that the construct is an annotation.
- An annotation can contain zero, one, or multiple elements.

9.4 Where Annotations Can Be Used

Annotations can be used above:

- classes,
- interfaces,
- methods,
- fields,
- local variables.

9.5 Examples

```
@Entity
public class Users {
}
```

```
@Entity(tableName = "USERS")
```

10. Spring Bean Life Cycle and Callbacks

The bean lifecycle describes when and how a bean is created, how it behaves during its life, and how it is destroyed. The Spring container manages the lifecycle of the bean.

10.1 Bean Life Cycle Process

1. Container starts.
2. Bean is instantiated.
3. Dependencies are injected.
4. Custom initialization method executes.
5. Bean remains ready for use.
6. On container shutdown, custom destroy method executes.

10.2 Initialization and Destruction Callbacks

- Initialization callback methods execute after dependency injection is completed.
- Their purpose is to verify bean property values, perform custom initialization, or wrap original bean behavior.
- Destruction callback methods execute when the IoC container is about to remove the bean.
- Their purpose is to release held resources or perform finalization work.
- Custom method names may be used instead of `init()` and `destroy()`.

11. Bean Configuration Styles

Spring Framework supports three styles of bean configuration.

11.1 Annotation-Based Configuration

Bean configuration can be done using annotations such as:

- `@Service`,
- `@Component`,
- `@Scope`.

11.2 XML-Based Configuration

Beans can be configured using Spring XML configuration files. In Spring MVC applications, XML-based configuration can also be loaded automatically through boilerplate code in `web.xml`.

11.3 Java-Based Configuration

Starting from Spring 3.0, beans can be configured using Java programs and annotations such as:

- `@Configuration`,
- `@ComponentScan`,
- `@Bean`.

12. Spring Boot

Spring Boot includes Spring Framework features and is widely used for building rapid Spring applications. It allows developers to focus more on application logic instead of spending excessive effort on environment and configuration setup.

12.1 Build Systems

Two recommended build systems are:

- Maven,
- Gradle.

They provide strong support for dependency management.

12.2 Dependency Management

Spring Boot provides a list of managed dependencies for each release. Developers usually do not need to specify a version for every dependency in the build configuration because Spring Boot configures dependency versions automatically based on the selected Spring Boot version.

12.2.1 Maven Dependency

For Maven configuration, the Spring Boot Starter Parent project is typically inherited to manage Spring Boot starter dependencies. The version is specified for the parent starter dependency, and other starter dependencies usually do not require explicit Spring Boot version numbers.

12.2.2 Gradle Dependency

For Gradle configuration, Spring Boot starter dependencies are imported directly into the `build.gradle` file. The Spring Boot parent dependency mechanism used by Maven is not required in the same way for Gradle.

13. Spring Boot Code Structure

There is no single fixed layout for Spring Boot projects, but several best practices are commonly followed.

13.1 Recommended Project Organization

A project can be divided:

- by layer, such as service layer, entity layer, and repository layer,
- by module, such as separating data layer and web layer,
- by feature.

13.2 Avoid Default Package

A class is in the default package when it does not include a package declaration. This is not considered good practice because Spring scans classes inside named packages and sub-packages based on annotations such as:

- `@ComponentScan`,
- `@EntityScan`,
- `@SpringBootApplication`.

13.3 Package Naming Convention

It is recommended to use Java's reverse-domain package naming convention.

13.4 Main Application Class Placement

The main application class should be placed in the root package and commonly use annotations such as:

- `@SpringBootApplication`,
- `@ComponentScan`,
- `@EnableAutoConfiguration`.

This allows Spring to scan all classes in the root package and sub-packages.

13.5 Common Structuring Approaches

Two common project structuring approaches are:

1. Structure by feature
2. Structure by layer

13.6 Example Main Application Class

```
@SpringBootApplication
public class MyApplication {
    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }
}
```

14. RESTful Web Services

RESTful APIs use HTTP methods to perform standardized operations on resources. Understanding these methods is essential for building robust APIs.

14.1 Common HTTP Methods

The main HTTP methods covered are:

- GET,
- POST,
- PUT,
- DELETE.

Other methods named are PATCH, HEAD, and OPTIONS.

14.2 Importance of HTTP Methods

Using the correct HTTP method helps in building robust and well-designed APIs that maintain consistency, interoperability, and efficiency between clients and servers.

14.3 GET Method

GET is used for retrieving data from the server. It is generally used when the client requests information without modifying server-side resources.

14.4 POST Method

POST is used for submitting data to the server for processing. Unlike GET, POST can cause modifications on the server.

14.5 PUT Method

PUT is used for updating or replacing an existing resource on the server. PUT is **idempotent**, which means that sending the same PUT request multiple times should result in the same final state.

14.6 DELETE Method

DELETE is used to remove a specified resource permanently from the server. Proper authorization, authentication, and error handling are important when implementing DELETE operations.

15. Rest Controller

A REST controller handles HTTP requests and produces HTTP responses, typically for RESTful web services. It works with request mappings, request bodies, path variables, and request parameters to build APIs.

16. Request Mapping

`@RequestMapping` is one of the most important annotations in Spring. It maps HTTP requests to handler methods of MVC and REST controllers.

In Spring MVC applications, the `DispatcherServlet` acts as the front controller and routes incoming HTTP requests to appropriate handler methods.

17. Request Body

Request body is the data sent in the body of an incoming HTTP request. It can be in various formats such as:

- JSON,
- XML,
- PDF,
- HTTP forms,
- and many other formats.

The `Content-Type` header tells the server what type of data is present in the request body.

17.1 @RequestBody

The `@RequestBody` annotation is used to get the body of an incoming request and deserialize the payload into a Java object or a map.

18. Path Variable

`@PathVariable` is used to retrieve data from the URL path. Placeholders in the request mapping URL are bound to method parameters annotated with `@PathVariable`. This allows dynamic values in the URL to be used directly inside handler methods.

19. Building Web Applications

Java is one of the most widely used languages for developing dynamic web applications. A web application is software that is accessed through a web browser and deployed on a web server.

19.1 Technologies Used in Web Applications

- Frontend technologies commonly include HTML, CSS, and JavaScript.
- Backend technologies can include Java, Python, PHP, and databases.
- Java technologies such as Servlet and JSP support web application development.
- Frameworks such as Spring and Spring Boot simplify development and reduce developer effort.

19.2 Need for Server-Side Technology

Static websites can be built using HTML and CSS, but dynamic websites require server-side technologies.

20. MVC Architecture

MVC stands for Model–View–Controller. It is an architecture that separates application components based on responsibility.

20.1 Components of MVC

- **Model:** Handles application data and business logic.
- **View:** Handles presentation or user interface logic.
- **Controller:** Receives requests from the view, gets required details from the model, and transfers them appropriately.

The view and model do not directly interact with each other.

21. Practical Task

21.1 Objective

Design and implement a small web application using Spring Boot that demonstrates RESTful API design, dependency injection, and layered architecture (controller, service, repository).

21.2 Suggested Problem Statements

- To-Do List Manager (create, update, delete, list tasks)
- Simple Calculator API (add, subtract, multiply, divide)
- Student Management System (CRUD operations on student records)

21.3 Functional Requirements

- Expose REST endpoints using `@RestController`.
- Use appropriate HTTP methods: GET, POST, PUT, DELETE.
- Accept input using `@RequestBody` and `@PathVariable`.
- Validate basic inputs and return meaningful responses.

21.4 Technical Requirements

- Use Spring Boot with Maven or Gradle.
- Organize code into layers: controller, service, repository, entity.
- Use Dependency Injection for wiring components.
- Maintain proper package structure (avoid default package).

21.5 Implementation Steps

1. Create a Spring Boot project using Spring Initializr.
2. Define the main application class with `@SpringBootApplication`.
3. Create entity/model classes.
4. Implement repository (in-memory or using a database).
5. Implement service layer for business logic.
6. Implement controller layer with REST endpoints.
7. Test APIs using Postman or curl.

21.6 Evaluation Criteria

- Correct use of HTTP methods and status codes.
- Clean separation of concerns (layered architecture).
- Proper use of annotations and dependency injection.
- Code readability and organization.

22. Quick Recap

This unit covers:

- Java frameworks and their purpose,
- Spring Framework basics and advantages,
- Dependency Injection,
- Inversion of Control,
- Aspect Oriented Programming,

- bean scopes,
- autowiring,
- annotations,
- bean lifecycle and callbacks,
- bean configuration styles,
- Spring Boot basics,
- dependency management and code structure,
- RESTful web services,
- `@RequestMapping`, `@RequestBody`, and `@PathVariable`,
- building web applications using MVC architecture.